

Project 3 (Teams of 3)

Simulating the Internet of Things through the Web of Things

Access → Find → Share → Compose

General Note

While this document presents a reference scenario for guidance, you are encouraged to design and implement your own Web of Things scenario. Any alternative concept, as long as it adheres to the same principles of interoperability, openness and composition through Web interfaces, is acceptable. An alternative scenario must be validated by the teacher before the first checkpoint (see Timeline) and must provide at least the same number of things, rules and interaction types as the reference scenario. The *API conventions* and the provided test script apply to the reference scenario; an alternative scenario must document its own conventions in its `README.md`.

Projects that go beyond the suggested example will be considered for a bonus grade; the eligible extensions are listed in the section *Bonus*.

Context

This project focuses on the application-level aspects of the Web of Things, deliberately omitting the network and hardware layers. No sensors or embedded devices are needed since all interactions are simulated through lightweight web services.

The aim is to explore how everyday things can be represented, exposed, and managed on the Web as if they were real connected objects. Through this simplified, software-only approach, the project highlights the essential principles of interoperability, openness, and composability that define the Web of Things, while remaining easy to test and extend without physical hardware.

Objective

The project aims to design and implement a Web of Things environment where virtual devices ('things') can interact through web-based interfaces. Each virtual thing exposes RESTful endpoints for its properties, actions, and events. These interfaces enable discovery through a registry, controlled access through access tokens, and composition into automated interactions among multiple things.

The final outcome is a functional demonstration of interoperability, real-time interaction, and secure coordination between distributed web-based objects.

Prerequisites and Starter Kit

This project builds on **Lab 3** (OpenAPI Specification) : describing a REST API with an OpenAPI contract, generating a Spring Boot server with `openapi-generator`, implementing a controller returning `ResponseEntity` objects with the proper status codes (200, 201, 404, 409), keeping the state in memory, and testing the API with `curl` or Postman.

The following aspects are new with respect to Lab 3, and are the core of the project :

- *several services calling each other* (things ↔ gateway), i.e. writing HTTP *client* code inside a server (e.g. Spring `RestClient`);
- *sub-resources and links* between resources (`/things/{id}/properties/{name}`);

- *real-time push* to a browser (Server-Sent Events, e.g. Spring `SseEmitter`);
- *token-based access control* (a filter or interceptor checking a header);
- *time-based behaviour* : periodic tasks (e.g. Spring `@Scheduled`) and timers that can be cancelled and restarted (e.g. Spring `TaskScheduler`);
- a small *web dashboard* (HTML and JavaScript `fetch`).

A starter kit for **Java/Spring Boot** is provided on Celene. It contains a minimal thing skeleton that registers itself with the gateway (`RestClient`), a Server-Sent Events example (`SseEmitter` on the server side, `EventSource` in the browser), a Bearer-token checking filter, a periodic task example (`@Scheduled`), a restartable timer (`TaskScheduler`), a minimal dashboard page, and a test script checking the mandatory routes and the *API conventions*.

Reference Scenario

The Computer Science department of Tours University aims to develop a simple and functional demonstrator consisting of three virtual things : a *Thermostat*, a *Lamp*, and a *Motion Sensor*. Each thing operates as an independent web service that exposes its properties, actions, and events through RESTful APIs.

The things are coordinated by a central *gateway service*, described in the section *System Design and Architecture*.

Minimal interface of each thing. Each thing must expose at least the following interaction affordances (the names are mandatory : together with the *API conventions* given below, they allow the gateway, the dashboard and the provided test script to rely on them) :

Thing	Properties	Actions	Events
Thermostat	<code>temperature</code> (read-only, simulated), <code>target</code> (°C), <code>mode</code> ∈ { <code>off</code> , <code>heat</code> , <code>eco</code> }	<code>setTarget</code>	<code>targetReached</code> , <code>propertyChanged</code>
Lamp	<code>on</code> (boolean), <code>brightness</code> (0–100)	<code>toggle</code> , <code>setBrightness</code>	<code>propertyChanged</code>
Motion Sensor	<code>lastMotion</code> (timestamp, read-only)	<code>simulateMotion</code>	<code>motion</code>

Simulation. In `heat` and `eco` modes, the thermostat regulates the temperature towards `target` : below it, the temperature rises by a fixed step at each tick (e.g. +0.2 °C/s); when it reaches `target`, the thermostat emits `targetReached` (only when the target is reached from below) and then holds the temperature at `target`; above it, the temperature drifts down to `target`, without event. In `off` mode, the temperature slowly drifts down towards an outside temperature (e.g. −0.05 °C/s down to 15 °C). Motion is triggered by calling the `simulateMotion` action (from the dashboard or with `curl`); random generation is optional. All simulation parameters (initial and outside temperatures, steps, rule delays) must be configurable, so that the whole scenario can be shown in a few minutes; an initial temperature below 19 °C (e.g. 17.5 °C) lets rule R2 be shown right away.

Description of a thing. Each thing describes itself at `GET /model`, following the Web Thing Model seen in the course (name, description, properties, actions, events), with a link to each of its resources, for instance :

```
"links": {"properties": "/properties", "setTarget": "/actions/setTarget"}
```

Its resources follow the course layout : `/properties`, `/properties/{name}`, `/actions/{name}`. Providing a W3C Thing Description instead is a bonus.

A web-based dashboard provides an integrated interface where users can :

- view the list of registered things and their descriptions,
- inspect the current state (properties) of each thing,
- modify settings such as temperature, brightness, or power,
- trigger actions like turning on the lamp or setting a target temperature,
- simulate motion,
- visualize live updates as things change state or emit events,
- and, if rule R4 is implemented (bonus), resume automation on a thing after a manual override.

The system must also implement a set of automation rules that combine multiple virtual devices and demonstrate composition across services.

The rules are evaluated by the gateway. **Rules R1 and R2 are mandatory ; R3 and R4 are bonus** (see the section *Bonus*). T_1 , T_3 and T_4 are configurable (default values : 60 s, 300 s, 600 s ; suggested values for the demo : 10 s, 30 s, 60 s).

Rule	Trigger	Effect	End / re-arming
R1 Motion-based lighting	event <code>motion</code>	lamp on	lamp off after T_1 without motion ; every new motion restarts T_1
R2 Temperature comfort	event <code>motion</code> while <code>temperature < 19°C</code>	thermostat mode = heat, <code>target = 19</code> (the lamp is switched on by R1)	on <code>targetReached</code> while mode = heat : mode = off
R3 Energy saving (<i>bonus</i>)	no motion during T_3	thermostat mode = eco, <code>target = 17</code>	on the next motion, before R2 : <code>target = 19</code> , mode = off ; then R2 applies
R4 Manual override (<i>bonus</i>)	any write or action sent by a client (dashboard, <code>curl</code>) through the gateway to the lamp or the thermostat	rules R1–R3 no longer act <i>on this thing only</i>	automation resumes after T_4 , or earlier if the user clicks “resume automation”

R1 illustrates event-driven and time-based control ; R2 multi-condition composition (an event combined with a state) ; R3 the absence of events as a trigger ; R4 the priority of user intent over background automation.

Rule semantics.

- The rules switch the lamp by writing its property `on` (PUT `.../properties/on`), never with `toggle`, which would switch it off if it is already on.
- On a `motion` event, the rules are applied in this order : end of R3 (if the thermostat is in `eco` mode), R1, R2. In `eco` mode, `targetReached` is ignored.

- T_3 is counted from the start-up of the gateway and restarted by every motion.
- The requests sent by the rules are not manual actions : the gateway sends them directly to the things, not through its own proxy, so they never trigger R4. Neither is `simulateMotion` : the motion sensor is not controlled by the rules.
- When an override ends, the thing keeps its current state ; the rules act on it again from the next event.
- Without R4, the rules may override a manual setting (e.g. R2 switching off a heating started by hand when `targetReached` is emitted).

System Design and Architecture

The system is composed of the three things described above, a central *gateway service* and a web dashboard. Each thing runs as a separate HTTP process on its own port.

The gateway acts as a directory and coordination point.

- **Registration.** At start-up, each thing registers itself with `POST /things`, with its token (see *Security*) and the body `{"id", "name", "baseUrl", "model"}`, where `model` is its description. The gateway answers 201, or 409 if the identifier is already used. A thing can unregister with `DELETE /things/{id}`.
- **Liveness (bonus).** The gateway periodically checks each registered thing (e.g. `GET /model` every 10s) and marks unreachable things as `offline` (field `"status": "online"|"offline"` in `GET /things`).
- **Discovery.** `GET /things` lists the registered things with a link to each one ; `GET /things/{id}` returns the description of a thing.
- **Proxy.** Reads, writes and actions go through the gateway :
 - `GET /things/{id}/properties` (all the properties),
 - `GET` or `PUT /things/{id}/properties/{name}`,
 - `POST /things/{id}/actions/{name}`,
 - `POST /things/{id}/automation/resume` (end of a manual override, if R4 is implemented).

The dashboard never calls a thing directly (Auth Proxy pattern of the course). If a thing does not answer, the gateway returns 502.

- **Events.** When its state changes, a thing notifies the gateway with `POST /events` (webhook). The gateway runs the automation rules and pushes the event to the connected browsers through the stream `GET /events/stream`.
- **API contract.** As in Lab 3, the gateway API must be described by an OpenAPI specification, in a file named `gateway.yaml`. Generating the server skeleton from this contract with `openapi-generator (-g spring)`, as done in Lab 3, is not required (bonus).

API conventions. They are common to all teams, and the provided test script checks them :

- identifiers and ports : gateway on port 8080 ; things `thermostat` (8081), `lamp` (8082) and `motion` (8083) ;
- reading a property returns `{"name": "brightness", "value": 80}` ; writing a property and invoking an action take the body `{"value": ...}` (no body for an action without input, such as `toggle`) ;

- status codes : 200 for a read, a write or an action; 201 (with a `Location` header) for a registration; 204 for an unregistration; 202 for `POST /events`; 400 for a missing field, an invalid or out-of-range value, or a write to a read-only property; 401 (see *Security*); 404 for an unknown thing, property or action; 409 for an identifier already registered; 502 for a thing that does not answer;
- every error has a JSON body with an explicit message : `{"status": 404, "error": "thing lamp is not registered"}`.

A web-based dashboard serves as the main user interface. It retrieves the list of registered things, displays their current state, and allows users to trigger actions such as changing a property value or invoking an operation. Whenever a thing's state changes, for example when the lamp turns on or the thermostat updates its temperature, the gateway broadcasts the update to all connected clients through a live event channel, keeping the dashboard synchronized in real time. Server-Sent Events are recommended for this one-way channel (see the course); WebSockets are accepted, provided the choice is justified in the report. The dashboard is served by the gateway itself, to avoid cross-origin (CORS) issues.

All exchanges rely on lightweight web formats such as JSON.

Security

- Every request to the gateway API (`/things`, `/events` and their sub-resources) must carry a token : `Authorization: Bearer <token>`. The dashboard files (`/`, HTML, JavaScript, CSS) are public : the page asks the user for a token and sends it with each API request.
- A single static token, defined in a configuration file, is sufficient; the things use it too, to register and to send their events.
- The gateway answers 401 when the token is missing or unknown. Every error response contains a JSON body with an explicit message, e.g. `{"error": "missing or invalid token"}`.
- Roles (`viewer` / `operator`) with 403 responses are a bonus.
- The browser `EventSource` API cannot send headers : for `/events/stream` only, the token may be passed as a query parameter (`?token=...`).

Technical Constraints

- Language : **Java with Spring Boot** (recommended : this is the environment of Lab 3, and the starter kit uses it) or **Node.js** (allowed, but not covered by the labs and without starter kit). All services of a team use the same language.
- Each thing and the gateway are separate processes listening on distinct ports; the whole system must start with a single command (e.g. a Maven multi-module project or a script launching `mvn spring-boot:run` for each service; `docker compose` is optional).
- In-memory state is sufficient (a `Map`, as in Lab 3) : no database is required.
- Dashboard : plain HTML/JavaScript is sufficient (with Spring Boot, static files placed in `src/main/resources/static` of the gateway are served automatically); a front-end framework is allowed but not expected.

Deliverables, Installation, and Execution

The final submission must include all source code, documentation, and materials required to reproduce the project. All components should be organized in a clear and documented codebase, hosted on a public or private Git repository (GitHub, GitLab, etc.), with concise installation and execution instructions.

The project must include :

- A documented **codebase** implementing the full system, including the gateway, virtual thing services, and dashboard, producing clear and well-structured outputs with explicit error messages (JSON body with the status codes of the *API conventions*).
- The **OpenAPI specification** of the gateway (`gateway.yaml`).
- A short **report** (2–3 pages) describing the architecture, design choices, discovery process, security mechanisms, and rule execution logic, organised in five sections : Access, Find, Share, Compose, Limits and possible improvements.
- A concise **demo video** (about 3 minutes) showing discovery, read/write operations, one action, real-time updates, and at least one automation rule. Suggested script : start of the services and registration → discovery → reading and writing a property → an action → real-time update on the dashboard → R1 → R2 → a request refused without token.
- An **installation guide** included in the main `README.md`, explaining how to install and run the system locally. It should mention the required environment and describe how to install dependencies, start all services, test the APIs, and reproduce the demo scenario with events and automation rules.

Submission deadline : Tuesday, October 6th, 2026, on Celene : link to the Git repository with a tag `v1.0`, the report (PDF) and the video (MP4, or a link).

Bonus

The following extensions can earn the bonus (capped at 3 points in total) :

- roles **viewer** / **operator** with 403 responses ;
- liveness checking of the registered things ;
- server skeleton generated from `gateway.yaml` with `openapi-generator` ;
- a W3C Thing Description for each thing, instead of the Web Thing Model ;
- content negotiation : HTML for a browser, JSON for a program (**Accept** header) ;
- HAL-compliant hypermedia links (`_links`, `_embedded`), e.g. with Spring HATEOAS ;
- a rule using an external Web API (e.g. outdoor temperature from a weather service) ;
- things that only accept requests coming from the gateway (shared gateway key) ;
- automated tests of the gateway API (e.g. JUnit tests with a client generated from `gateway.yaml`, as in Lab 3).